# A Weighted Dependency Solver for Diagnosing and Repairing Minecraft Modpacks

Dylan OBrien
Wentworth Institute of Technology
School of Computing and Data Science
Boston, Massachusetts, United States
obriend7@wit.edu

Weije Pang
Wentworth Institute of Technology
School of Computing and Data Science
Boston, Massachusetts, United States
pangw@wit.edu

## Abstract

Minecraft modpacks combine independently developed modifications whose game-version, loader, version, dependency, and conflict constraints must agree. Existing launchers provide installation, update, and partial dependency assistance, but users commonly resolve invalid custom configurations through manual trial and error. This paper presents a weighted dependency solver that diagnoses Fabric/Modrinth metadata and searches for an explainable, minimum-disruption repair plan. The system supports dependency additions, version changes, removals, duplicate cleanup, cascading repairs, two preference profiles, offline metadata caches, and a Tkinter interface. Evaluation used 158 deterministic cases from 89 source families, including Modrinth-derived manifests, injected variants, dependency-dense topologies, cascading cases, and search-stress cases. A one-pass baseline repaired 91 of 104 repairable cases (87.5%), while both weighted profiles repaired 104 of 104. Mean original-mod preservation increased from 94.93% for the baseline to 97.46% under default weights and 98.42% under preservation-focused weights. The weighted solver also repaired 11 of 11 cascading cases, correctly identified 13 of 13 no-solution cases, and agreed with an independent exhaustive oracle on 12 of 12 comparable optimal plans. Results demonstrate the value of complete-plan search, while remaining limited to metadata-level compatibility rather than guaranteed launch success.

## CCS Concepts

• **Software and its engineering** → **Software configuration management and version control systems**; *Software libraries and repositories*; • **Computing methodologies** → *Search methodologies.*

## Keywords

Minecraft modpacks, dependency resolution, weighted search, software repair, compatibility analysis, Modrinth

## 1 Introduction

Minecraft is an open-world sandbox game in which players gather resources, explore procedurally generated environments, survive hostile creatures, and build structures or machines from blocks. Its lack of a single required play style has supported creative, survival, multiplayer, educational, and community-developed experiences since its early public release in 2009 and full release in 2011 [6]. In April 2025, Guinness World Records listed Minecraft as the best-selling video game, with more than 350 million units sold [5]. This scale and longevity make compatibility within its community software ecosystem a practical software-engineering problem rather than a small hobby example.

Minecraft: Java Edition is commonly extended through third-party modifications, or *mods*, that add mechanics, content, interfaces, performance improvements, or entirely new styles of play. A *modpack* combines many independently developed mods into one repeatable configuration. Packs can range from a few utilities to hundreds of components maintained by different authors and released on different schedules. Mojang does not provide official support for third-party Java mods, so players and pack authors rely on mod loaders, launchers, repositories, and community troubleshooting processes [11].

The components of a modpack must agree on the Minecraft version, mod loader, selected mod versions, required dependencies, and declared incompatibilities. A pack may become invalid when a dependency is omitted, an incompatible version is selected, conflicting mods are combined, or one correction introduces additional transitive requirements. Diagnosing these problems becomes increasingly difficult as the number of mods, candidate versions, and dependency relationships grows.

Launchers simplify installation, but a user who creates or modifies a pack may still need to determine which mod should be added, upgraded, downgraded, or removed. A locally reasonable action can also reveal another problem. For example, changing a mod version may resolve a game-version mismatch while introducing a new dependency, which may then conflict with another selected mod. A useful repair tool must therefore evaluate complete action sequences rather than only the first visible issue.

The **goal of this research** is to design, implement, and evaluate an explainable solver that transforms an invalid Fabric/Modrinth modpack description into a metadata-compatible configuration while minimizing disruption to the user's original choices. The project has five concrete objectives:

(1) normalize multiple input forms into one reproducible internal representation;
(2) construct a dependency graph and identify game, loader, dependency, conflict, and uniqueness violations;
(3) generate domain-specific repair actions and compare complete multi-step plans using configurable costs;
(4) evaluate repair success, preservation, correctness, and runtime against a deterministic one-pass baseline; and
(5) present the selected repair and its reasoning through readable explanations, exports, and a desktop interface.

The resulting system constructs a dependency graph, reports compatibility issues, generates candidate actions, and performs

bounded best-first search for a valid configuration with minimum cumulative disruption. Adding a required library receives a low cost; changing an explicitly selected mod is more disruptive; removal and environment changes receive larger penalties. A second profile increases the cost of changing or removing the user's original selections.

The implementation also includes metadata normalization and caching, a deterministic one-pass baseline, explanation generation, reproducible evaluation tools, and a Tkinter interface. Supported inputs include project JSON, basic `.mrpack` manifests, Modrinth URLs or project lists backed by cached metadata, built-in cases, and the evaluation corpus. The application returns a repair plan; it does not download mod JARs, launch Minecraft, or automatically rewrite a pack.

This study addresses four research questions:

(1) **RQ1:** Does weighted complete-plan search repair more invalid configurations than a one-pass baseline?
(2) **RQ2:** Does weighted repair preserve more of the original mod selection?
(3) **RQ3:** How does runtime change with pack and dependency-graph complexity, and how does bounded search behave as state exploration increases?
(4) **RQ4:** Can the system correctly handle and structurally explain cascading, no-solution, and oracle-verifiable cases?

The main contributions are a Minecraft-specific compatibility model, a configurable weighted repair algorithm, independently grounded test cases, real-manifest and controlled-complexity evidence, and an end-user interface for explainable repair planning. The research contribution is not a replacement launcher; it is an experimentally evaluated repair-planning layer that makes global dependency decisions visible and configurable.

## 2 Background and Related Work

### 2.1 Minecraft Modpacks as Software Configurations

A modpack is more than a list of filenames. Each selected mod normally corresponds to one specific release of one project, and that release declares the game versions and loaders it supports. It may also identify required, optional, incompatible, or embedded projects. A single project can publish many releases, so replacing one selected version may change both its environment support and its outgoing dependency relationships. Modpack repair therefore resembles software configuration management: the system must choose a consistent set of component versions rather than independently fixing isolated files.

Mod loaders provide the runtime mechanisms that allow mods to participate in the game. This project focuses on Fabric because it offers a well-defined metadata ecosystem and because limiting the loader reduced the number of unrelated packaging differences during evaluation. Modrinth was selected as the primary repository because its API and `.mrpack` specification expose project, version, environment, and dependency fields in machine-readable form [7, 9]. The restriction improves experimental control, but it also limits how directly the results generalize to Forge, NeoForge, Quilt, or CurseForge metadata.

Three properties make the problem difficult. First, dependencies are often *transitive*: a selected mod requires a library that itself requires another library. Second, candidate versions can create global tradeoffs; the version that fixes one issue may introduce another. Third, multiple valid configurations may exist, and users generally prefer a repair that preserves their original feature selection. These properties motivate graph-based checking followed by cost-aware search.

### 2.2 Dependency Solving and Optimization

Software dependency resolution selects component versions that satisfy mandatory requirements while avoiding conflicts. In nontrivial component models, mutually exclusive versions and explicit conflicts make dependency solving computationally hard; incomplete greedy procedures may fail even when a valid configuration exists [1]. This motivates separating compatibility checking from a solver capable of considering global combinations.

Validity alone does not determine which solution is most desirable. Multiple configurations may satisfy all hard constraints while differing in removals, version changes, age, duplication, or security. PacSolve demonstrates how dependency constraints can be combined with customizable optimization objectives through Max-SMT [13]. The present project follows the same hard-constraint/soft-preference distinction, but uses a smaller deterministic best-first search so that the output is an ordered sequence of domain-specific repair actions.

### 2.3 Minecraft Metadata and Existing Tools

Modrinth version metadata records supported game versions, loaders, and dependencies classified as required, optional, incompatible, or embedded [7]. Modrinth modpacks are distributed as ZIP-based `.mrpack` archives whose root contains a `modrinth.index.json` manifest listing the environment and downloadable files [9]. These fields provide useful static evidence, although they cannot capture every runtime interaction.

The official Minecraft Launcher does not provide mod support; Mojang directs mod-related failures to third-party tooling and communities [11]. The Modrinth, CurseForge, and Prism applications provide broader instance installation and management, version filtering, updates, and manual troubleshooting workflows [2, 3, 8, 10, 14, 15]. packwiz automatically detects listed dependencies while authoring a pack and warns about game-version mismatches [12]. Narrow utilities such as ezMMCC inspect potentially conflicting class modifications rather than solving metadata constraints [4]. Table 1 summarizes the documented scope of these tools. The comparison is qualitative; it is not an external performance benchmark.

Existing launchers are more complete operational products than the proposed system. The research gap is narrower: the reviewed tools do not publicly document the same combination of complete multi-step repair search, configurable disruption costs, preservation-oriented optimization, cascading issue handling, and explanations of rejected alternatives.

**Table 1: Qualitative comparison of documented tool capabilities. "Not doc." means the reviewed public documentation did not describe the capability, not that it is impossible internally.**

| Tool | Primary role | Dependency assistance | Existing-pack diagnosis | Multi-step search | Weighted preferences | Alternative explanation |
|---|---|---|---|---|---|---|
| Minecraft Launcher | Game installation and launch | Not doc. | Not doc. | Not doc. | Not doc. | Not doc. |
| Modrinth App | Modpack installation and instance management | Partial | Partial | Not doc. | Not doc. | Not doc. |
| CurseForge App | Profile and modpack management | Partial | Partial | Not doc. | Not doc. | Not doc. |
| Prism Launcher | Multi-source instance management | Manual/partial | Partial | Not doc. | Not doc. | Not doc. |
| packwiz | Modpack authoring and dependency assistance | Yes | Partial | Not doc. | Not doc. | Not doc. |
| ezMMCC | Narrow class-conflict detection | N/A | Partial | Not doc. | Not doc. | Not doc. |
| Proposed solver | Metadata diagnosis and weighted repair planning | Yes | Yes | Yes | Yes | Yes |

# 3 System Design and Methodology

## 3.1 Pipeline and Data Model

The system imports a configuration, normalizes metadata, builds a dependency graph, checks hard constraints, generates candidate repairs, searches for compatible states, and then produces short and technical explanations. The same typed models, graph builder, and checker are reused by the baseline, weighted solvers, GUI, and evaluation scripts.

A modpack state is represented as

$$S = \langle M, V_{\mathrm{MC}}, L, U \rangle, \tag{1}$$

where $M$ is the set of selected project-version pairs, $V_{\mathrm{MC}}$ is the Minecraft version, $L$ is the loader, and $U$ is the normalized universe of available projects and versions. The selected configuration forms a directed graph $G = (N, E)$ whose edges are labeled required, optional, incompatible, or embedded.

A state is compatible when all hard constraints hold:

$$\begin{aligned}\mathrm{Compatible}(S) = {} & C_{\mathrm{game}}(S) \wedge C_{\mathrm{loader}}(S) \\ & \wedge C_{\mathrm{required}}(S) \wedge C_{\mathrm{conflict}}(S) \wedge C_{\mathrm{unique}}(S).\end{aligned} \tag{2}$$

These predicates require selected versions to support the chosen game and loader, require mandatory dependencies, prohibit incompatible selections, and prevent duplicate versions of one project. The checker returns structured issues with severity, affected mods, expected and observed values, and dependency chains.

## 3.2 Dependency Graph Construction and Use

For each state, the implementation constructs a labeled directed graph

$$G(S) = \left(N, E_{\mathrm{req}} \cup E_{\mathrm{opt}} \cup E_{\mathrm{inc}} \cup E_{\mathrm{emb}} \cup E_{\mathrm{game}} \cup E_{\mathrm{loader}}\right). \tag{3}$$

The node set contains project nodes, release nodes, Minecraft-version nodes, and loader nodes. A project-to-version edge records that a release belongs to a project. Separate version-to-environment edges record supported Minecraft versions and loaders. Dependency edges originate at a specific release because relationships can change between releases. For relationship type $t$, an edge is inserted when normalized metadata declares that relationship:

$$(u, v) \in E_t \iff \mathrm{metadata}(u) \text{ declares relation } t \text{ to } v. \tag{4}$$

Nodes corresponding to the current Minecraft version, loader, and selected releases are marked as selected. For a required edge from selected release $u$ to target $v$, the checker reports a missing dependency when the target project or target release is not selected:

$$\mathrm{Missing}(S) = \{(u, v) \in E_{\mathrm{req}} : u \in M \wedge v \notin \mathrm{Sel}(S)\}. \tag{5}$$

Likewise, an incompatible edge becomes an active hard conflict only when both endpoints are represented by selected components:

$$\mathrm{ActiveConflict}(S) = \{(u, v) \in E_{\mathrm{inc}} : u, v \in \mathrm{Sel}(S)\}. \tag{6}$$

The checker traverses outgoing edges from every selected release, verifies required targets, emits warnings for absent optional targets, reports active incompatibilities, and tests whether support edges reach the selected game and loader nodes. Dependency chains are recovered by following required edges and recording visited nodes, which prevents cycles from causing infinite traversal. The graph is rebuilt after every candidate action because changing a release changes its outgoing edges. This repeated construction is central to cascading repair: a newly added dependency can expose a second missing dependency or an incompatibility that was not present in the previous state.

The Python implementation uses NetworkX's `DiGraph` for labeled nodes and edges, Pydantic models for validated metadata and state objects, and standard dictionaries for project/version lookup. Selected configurations are copied before mutation, and a canonical sorted state key allows the search to recognize equivalent configurations. The same graph builder and checker are called by the baseline, weighted search, GUI, and evaluation scripts so that all components apply the same compatibility rules.

## 3.3 Repair Actions and Objective

A repair plan is an ordered sequence $P = \langle a_1, \ldots, a_k \rangle$, and each action transforms the current state as $S_i = T(S_{i-1}, a_i)$. Supported actions add dependencies, upgrade or downgrade dependencies and selected mods, remove selected mods, and resolve duplicate selections. Table 2 lists the default costs.

The plan cost and optimization objective are

$$C(P) = \sum_{i=1}^{k} w(a_i), \qquad P^* = \arg\min_{P} C(P) \tag{7}$$

subject to $\mathrm{Compatible}(T(S_0, P))$. The preservation-focused profile keeps dependency costs unchanged but raises selected-mod

**Table 2: Default repair-action costs.**

| Action | Cost | Action | Cost |
|---|---|---|---|
| Add dependency | 1 | Upgrade dependency | 2 |
| Downgrade dependency | 3 | Upgrade selected mod | 4 |
| Downgrade selected mod | 5 | Remove selected mod | 10 |
| Change Minecraft | 20 | Change loader | 25 |

upgrade, downgrade, and removal costs from 4, 5, 10 to 5, 6, 20. Raw costs are interpreted only within a profile because the scales differ.

## 3.4 Weighted Search, Baseline, and Explanations

The solver performs bounded deterministic best-first search. Each frontier item contains a configuration, the actions used to reach it, and cumulative cost. Python's `heapq` module implements the priority queue. The ordering used for search and deterministic tie-breaking is

$$\pi(S_i) = (C(P_i), -R_i, D_i, |P_i|, V_i, K_i)\,, \tag{8}$$

where $R_i$ is the number of original mods preserved, $D_i$ is the number removed, $V_i$ is the number whose versions changed, and $K_i$ is the canonical state key. Lower tuples are expanded first. Cost is the primary objective; the remaining fields make equal-cost choices stable and preservation-aware.

For each incompatible state, candidate actions are generated from the current structured issues. Missing-dependency issues produce additions or dependency-version changes; game or loader mismatches produce alternative releases of the affected project; conflicts produce version changes or removals; and duplicate selections produce deterministic cleanup. Each action creates a copied state, after which the graph and compatibility report are recomputed. A dictionary stores the best known cost for every canonical state, so a repeated configuration is discarded unless it was reached more cheaply. This permits non-monotonic cases in which one repair resolves an issue while revealing another.

Search stops when a compatible state is removed from the frontier, when the requested number of solutions is found, or when a configured bound is reached. Default safeguards allow at most six repair actions, 5,000 expanded states, and ten seconds. With non-negative action costs and no bound reached, expanding states by cumulative cost gives the same minimum-cost property as uniform-cost search over the generated repair-state graph. The claim is limited to the generated candidates and modeled metadata; it is not a proof that every possible ecosystem repair was represented.

The comparison baseline obtains the checker's initial suggestions and applies each executable suggestion once, in order. It selects the first compatible candidate version, does not regenerate suggestions after intermediate actions, and does not backtrack or compare complete plans. This models a deterministic local-repair workflow rather than a random strawman.

For a weighted result, the explanation layer reports the initial causes, dependency chains, action sequence, costs, rejected alternatives, final compatibility, runtime, and states expanded. The GUI exposes the same information and can export readable text or JSON. Explanation quality in this study is evaluated structurally; no human-readability study was conducted.

**Table 3: Composition of the 158-case main corpus.**

| Source group | Cases |
|---|---|
| Synthetic regression | 19 |
| Modrinth-derived controls | 11 |
| Modified Modrinth-derived | 13 |
| Legacy custom scale | 39 |
| Dependency-dense topology | 48 |
| Cascading and search stress | 28 |
| **Total** | **158** |

# 4 Evaluation Datasets

The corpus is not a machine-learning dataset: no model is trained and no train/test split is used. Each item is a deterministic configuration with selected versions, candidate metadata, expected issues and solver outcome, provenance, ground-truth method, and graph-complexity measurements. Cases are grouped by *source family*; variants derived from one base configuration remain in the same family so that a single original design does not appear to be many independent examples.

The synthetic regression group contains small, deliberately constructed cases that isolate one expected behavior, such as a missing dependency, duplicate selection, loader mismatch, version mismatch, or conflict. They are called regression cases because they are rerun after implementation changes to verify that previously working behavior remains correct. These cases are not intended to imitate the full variety of public packs; their role is to make failures easy to localize and reproduce.

The cases represent 89 source families. Their expected outcomes are 41 already-compatible cases, 104 repairable invalid cases, and 13 expected no-solution cases. Size categories contain 88 small, 28 medium, 22 large, and 20 huge cases. The largest controlled case has 250 selected mods, 475 required edges, 503 total edges, and required-dependency depth 16.

The scored real-data subset contains two reduced Modrinth controls and four reduced variants, plus nine complete-manifest controls and nine paired variants. A complete-manifest case covers the full Modrinth file list through normalized cached metadata; the project does not redistribute an official pack archive or mod JARs. The collector obtained 20 complete manifests, but only nine controls met the automated scoring criteria. Other collection and normalization outcomes were quarantined rather than treated as broken public packs.

Controlled cases provide structures that are difficult to obtain reliably from a small real-pack sample. Dense generators produce chains, layered directed acyclic graphs, shared-library fan-in, and clustered modules across size categories. Generation parameters determine selected-mod count, required-edge count, optional/conflict edges, and chain depth, allowing graph traversal and reconstruction cost to be varied without changing the checker semantics.

Cascading cases include dependency chains, version changes that introduce new dependencies, dependency additions that create conflicts, global candidate choices, cycles, and unsatisfiable alternatives. Search-stress cases exercise equal-cost ties, profile-sensitive decisions, candidate branching, and no-solution outcomes.

Real-derived faults are created through reversible injections. A known-valid control can be modified by removing one required dependency, selecting a release that does not support the configured game or loader, duplicating a project selection, or inserting a controlled incompatibility. The expected repair is then established from the untouched control rather than copied from the weighted solver's output. This design provides realistic metadata structure while retaining a known reason for the failure.

Ground truth is independent of the weighted solver: 50 cases use known valid controls, 91 use reversible injections, and 17 use bounded exhaustive reference enumeration. Known repairs are replayed through the checker, and expected labels remain separate from observed solver outputs. All 158 cases passed automated validation. Sixty-two review packets were generated, but none had completed human judgment; therefore the paper makes no claim of human-rated explanation quality or publication review.

## 5 Experimental Design

Each case was evaluated with three systems: the one-pass baseline, the weighted solver with default costs, and the weighted solver with preservation-focused costs. Repair success required a case marked repairable to end in a checker-valid configuration. For repairable set $Q$, success rate is

$$\text{SuccessRate} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\text{Compatible}(S_q^{\text{final}})]. \tag{9}$$

For original project set $M_0$ and repaired project set $M_f$, preservation is

$$\text{Preserve}(S_0, S_f) = \frac{|M_0 \cap M_f|}{|M_0|}. \tag{10}$$

Mean preservation was calculated among successful repairs. To avoid the baseline's smaller success denominator, a second measure counted full-preservation repairs over all 104 repairable cases. Additional measures included removals, runtime, states expanded, cascading-repair success, no-solution correctness, oracle agreement, and structured explanation completeness.

We also recorded graph structure separately from pack size. Edge density was the number of required dependency edges divided by the number of selected mods. Maximum dependency depth was the longest acyclic path through required edges after cyclic strongly connected regions were treated as units. Connected components, branching, and cycle counts were also retained. These measures distinguish a large collection of mostly independent mods from a smaller but highly connected configuration.

Runtime measured algorithm execution only. Each operation received one unmeasured warm-up and three measured repetitions using `time.perf_counter()`, with the median reported and minimum/maximum samples retained. The frozen environment used Python 3.14.3 on 64-bit Windows 11. Family-clustered 95% confidence intervals were estimated by resampling all 89 source families 2,000 times with a fixed seed. These intervals quantify variation within the controlled corpus; they do not imply random sampling from the full Minecraft ecosystem.

A separate five-case supplement targeted 32, 128, 212, 500, and 750 expanded states. It is not part of the main corpus and is intended only to demonstrate bounded search-state growth.

## 6 Results

### 6.1 Repair Success and Preservation

Table 4 summarizes the main outcomes. The baseline repaired 91 of 104 cases, while both weighted profiles repaired all 104, a 12.5-percentage-point increase. The family-clustered 95% interval for baseline success was 81.7–92.9%; both weighted intervals were 100–100% because every repairable case succeeded.

Full-preservation repair increased from 71/104 for the baseline to 88/104 under default weights and 92/104 under preservation-focused weights. Their family-clustered 95% intervals were 58.9–77.9%, 78.0–92.2%, and 82.5–95.4%, respectively. The two weighted profiles selected different plans in four profile-sensitive cases: the default profile removed one mod, while the preservation profile performed three version changes and retained every original mod. This demonstrates that the cost profile affects behavior rather than only the reported score.

Figure 1 localizes the baseline failures. It repaired none of six duplicate-version cases and only three of eleven cascading cases. It also missed five of 51 missing-dependency instances, four of 32 Minecraft-version mismatch instances, and one of 21 hard-conflict instances. The weighted solver repaired every repairable case in each category. Categories overlap when a case contains multiple issues.

### 6.2 Cascading, Oracle, and No-Solution Cases

The weighted solver repaired all 11 repairable cascading cases, compared with 3/11 for the baseline. Successful weighted cascading plans averaged 2.18 actions and reached a maximum depth of five. Every cascading case changed issue type after an intermediate action, and one temporarily increased the number of active errors.

A representative dependency-chain case illustrates the difference between local and complete-plan behavior. The initial selected mod required dependency $d_1$, but only $d_1$ was visible in the first compatibility report. After $d_1$ was added, its metadata exposed missing dependency $d_2$; adding $d_2$ then exposed $d_3$. The weighted solver produced

$$S_0 \xrightarrow{+d_1} S_1 \xrightarrow{+d_2} S_2 \xrightarrow{+d_3} S_3, \qquad C(P) = 1 + 1 + 1 = 3, \tag{11}$$

with $S_3$ compatible. The one-pass baseline applied only the initial suggestion and stopped at $S_1$, which remained invalid because it did not regenerate actions. This example shows why repeated graph reconstruction and checking are algorithmically necessary rather than only an implementation detail.

Seventeen small cases were exhaustively enumerated by an independent reference oracle. Twelve produced comparable optimal repair plans, and the weighted solver matched all twelve. The remaining five were exhaustively verified no-solution cases. Across the full corpus, all 13 expected no-solution outcomes were reported

**Table 4: Main solver comparison. Mean preservation is calculated among successful repairs; full-preservation counts use all 104 repairable cases as a common denominator. Runtime is the median across all 158 cases.**

| Method | Repairs | Success | Full preservation | Mean preservation | Mean removals | Median runtime (ms) | Failures |
|---|---|---|---|---|---|---|---|
| Baseline | 91/104 | 87.5% | 71/104 | 94.9% | 0.220 | 1.628 | 13 |
| Weighted default | 104/104 | 100.0% | 88/104 | 97.5% | 0.154 | 3.601 | 0 |
| Preservation-focused | 104/104 | 100.0% | 92/104 | 98.4% | 0.115 | 3.683 | 0 |

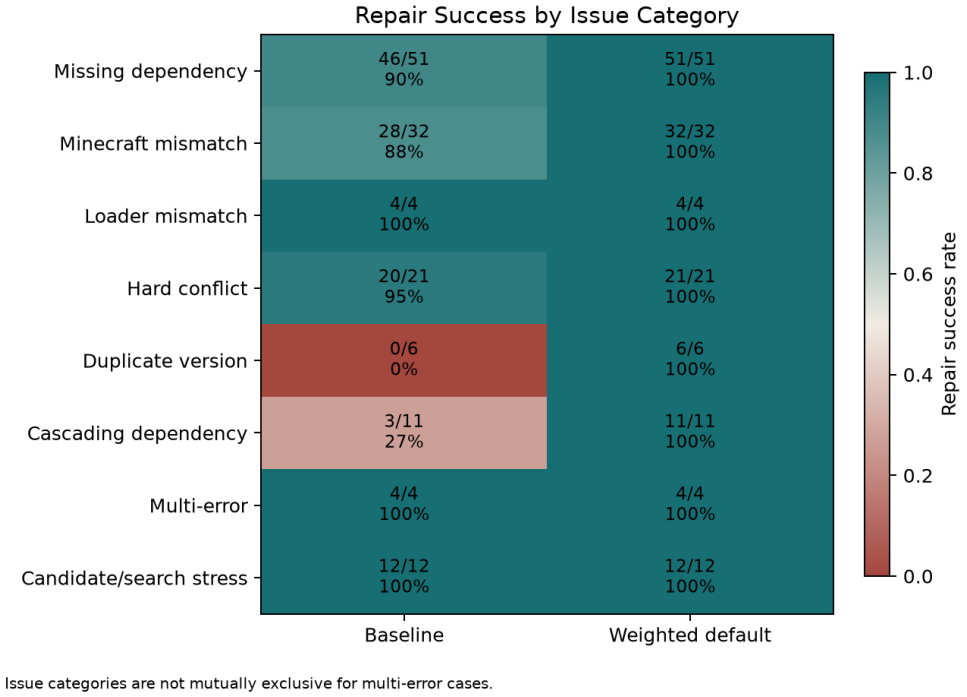


**Figure 1: Repair success by issue category. A case may contribute to multiple rows.**

correctly. Automated checks found all required structured explanation fields, dependency-chain content, cascading-step content, and global-plan rationale; these are structural checks rather than human readability scores.

## 6.3 Runtime and Scaling

The weighted solver's overall median runtime was approximately 3.6–3.7 ms, compared with 1.6 ms for the one-pass baseline. By pack size, weighted-default median runtime increased from 0.963 ms for small cases to 8.762 ms for medium, 16.961 ms for large, and 31.126 ms for huge cases; all repairable cases in each group succeeded. Figure 2 shows that runtime generally rises with selected-mod count and required-edge count, while the main corpus typically requires only two expanded states. Thus, the dense cases primarily stress metadata traversal, graph construction, and repeated checking rather than broad search.

A separate search experiment isolated state growth. The solver found valid plans at 32, 128, and 212 states with median runtimes of 12.638, 75.544, and 122.667 ms. It reached configured limits at 500 and 750 states after 314.540 and 977.145 ms. Those limit-reached cases retain independently known valid repairs; therefore the result demonstrates bounded resource behavior, not unsatisfiability.

# 7 Discussion

**RQ1: Repair effectiveness.** Complete-plan search was more reliable than the one-pass baseline on this corpus. The largest difference came from cases requiring re-evaluation, alternative choices, or duplicate cleanup. The baseline was effective for many direct errors, including all loader mismatches and most simple missing dependencies, but it could not reconsider an early choice or generate a newly required second step.

**RQ2: Preservation.** Both weighted profiles removed fewer original mods and produced more fully preserving repairs. The preservation-focused profile made a measurable difference only when cases contained an explicit tradeoff: it accepted several version changes to avoid one removal. This supports configurable

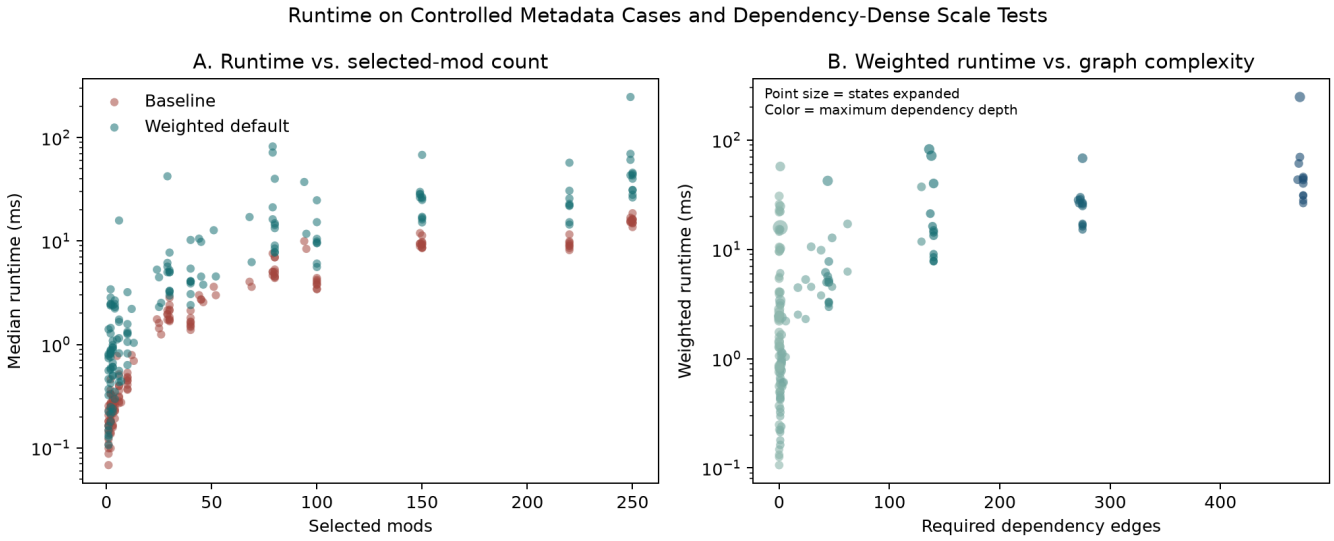


**Figure 2: Runtime on the main controlled metadata corpus. Panel A compares baseline and weighted runtime against selected-mod count. Panel B relates weighted runtime to required-edge count; point size represents states expanded and color represents maximum dependency depth.**

weights, but it also shows that profile evaluation requires cases where objectives genuinely compete.

**RQ3: Performance.** Runtime increased with pack and graph size, but remained tens of milliseconds at the median even for the large and huge controlled groups. The main corpus did not heavily stress search branching; the separate bounded experiment showed approximately increasing cost from 32 through 750 states and demonstrated the behavior of configured limits. These results support interactive metadata analysis, not a claim of ecosystem-wide or worst-case scalability.

**RQ4: Complex outcomes and explanations.** Cascading success, no-solution correctness, and oracle agreement show that the solver can do more than emit local suggestions. It can follow issue transitions, reject invalid candidates, and produce complete plans consistent with independently established outcomes. However, explanation evaluation was automated and structural; human understandability remains future work.

Complete-manifest cases demonstrate that the import and normalization pipeline can operate on real Modrinth metadata. The modified real-derived cases provide controlled faults on those structures. Dense topology, cascading, and search cases provide the interactions needed to distinguish algorithms. Consequently, the 12.5-point baseline difference should be attributed to the full mixed corpus, not solely to the complete Modrinth subset.

Compared with current launchers, the proposed system is intentionally narrower. Modrinth, CurseForge, and Prism install and manage playable instances, while packwiz supports production-oriented pack authoring. This project instead explores a repair-planning layer that could complement such tools. The qualitative comparison supports a novelty claim about the combination of features, but not a performance claim against external products.

## 7.1 Practical Interpretation

The cost values should be interpreted as an explicit preference policy rather than as universal measurements of inconvenience. Their purpose is to make the solver's behavior predictable: adding a missing library is normally preferred to replacing a chosen content mod, and either is preferred to changing the whole environment. A pack author can inspect the returned action sequence and rejected alternatives, then decide whether the encoded priorities match the intended experience. The preservation-focused profile demonstrates that the same hard compatibility rules can support a different user preference without rewriting the checker.

The separation between graph checking and repair search also supports future integration. A launcher could supply normalized metadata and present the returned plan before downloading files; a pack-authoring tool could use the checker during editing; and a debugging workflow could combine static metadata evidence with launch or crash evidence. In each case, the solver remains advisory until the user approves changes. This is important because metadata compatibility is necessary but not sufficient for runtime correctness.

# 8 Threats to Validity and Limitations

**Construct validity.** Checker compatibility is based on declared metadata. A plan that satisfies those constraints may still fail because of undeclared conflicts, mixin behavior, configuration interactions, side-specific assumptions, or runtime defects. Preservation measures retained project selections, not user-perceived feature equivalence.

**Internal validity.** The weighted solver and reference oracle share the compatibility checker, although the oracle does not reuse weighted search or its candidate ordering. A checker defect could therefore affect both. This risk is reduced through direct fixtures,

reversible injections, trace replay, legacy regression tests, and 353 passing normal tests plus a marked stress test.

**External validity.** The corpus is Fabric/Modrinth-focused. Only 24 scored cases are Modrinth-derived. Eighteen use complete manifests; the difficult topologies are controlled data. The 89 source families reduce duplicate influence, but they are not a random sample of all modpacks. Family-clustered intervals describe this corpus only. Repository metadata can also change after collection, so results describe the frozen cache used for evaluation rather than the current state of every public project.

**Conclusion validity.** Runtime samples were collected on one Windows computer and are appropriate for comparing methods within that environment, not for predicting exact performance on other hardware. Many main-corpus cases expand only a few search states, so the separate bounded supplement is needed to show broader state growth. The supplement is intentionally small and demonstrates resource-limit behavior rather than a full empirical complexity law.

**Review and reproducibility.** All 158 cases passed automated validation, but the 62 queued manual reviews were not completed. No human explanation study was performed. Complete-manifest cases remain pending publication review. Sixteen additional collection or normalization outcomes were conservatively quarantined. The evaluation snapshot records manifest hashes, commands, timing settings, and environment details, but the archived repository did not contain a verifiable final Git commit hash.

The tool also does not install files, rebuild `.mrpack` archives, inspect crash logs, or support Forge, NeoForge, Quilt, or CurseForge metadata. These are potential extensions rather than claims of the present implementation.

## 9 Conclusion

This capstone developed an end-to-end system for diagnosing and repairing Minecraft modpack metadata through weighted complete-plan search. The system models game, loader, version, dependency, conflict, and uniqueness constraints; evaluates candidate actions after every transition; supports configurable preservation preferences; and exposes the result through explanations, exports, and a GUI.

On 158 validated cases, the weighted solvers repaired all 104 repairable configurations, compared with 91 for the one-pass baseline. They preserved more original selections, repaired all cascading cases, correctly reported all expected no-solution cases, and matched every comparable oracle-verified optimum. Runtime remained suitable for interactive analysis on the tested corpus, while a separate supplement demonstrated bounded behavior through 750 expanded states.

The results support weighted repair planning as a useful complement to existing launcher and authoring workflows. Future work should validate repaired packs through actual launches, complete human review and explanation studies, extend metadata semantics and loader ecosystems, ingest crash evidence, and optionally generate repaired manifests after user confirmation.

## Acknowledgments

*[Replace this note with acknowledgments for advisor, reviewers, or contributors, or remove the section if it is not required.]*

## Ethics and Privacy Statement

This project evaluates public or deliberately generated software metadata and does not involve human participants or private user information. No mod JAR files are collected or redistributed. Public release should retain provenance records and follow applicable platform terms and metadata licenses. The system's recommendations should be treated as metadata-level guidance rather than a guarantee of safe or successful game execution.

## References

[1] Pietro Abate, Roberto Di Cosmo, Georgios Gousios, and Stefano Zacchiroli. 2020. Dependency Solving Is Still Hard, but We Are Getting Better at It. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 547–551. doi:10.1109/SANER48275.2020.9054837

[2] CurseForge. n.d.. Installing Minecraft Mods and Modpacks. Accessed 2026-07-21. https://support.curseforge.com/support/solutions/articles/9000196984-installing-minecraft-modpacks

[3] CurseForge. n.d.. Minecraft Modpacks: Installation and Launch Issues. Accessed 2026-07-21. https://support.curseforge.com/support/solutions/articles/9000196081-minecraft-troubleshooting

[4] DevilGlitch. n.d.. ezMMCC: Minecraft Mod Compatibility Checker. Accessed 2026-07-21. https://github.com/DevilGlitch/ezMMCC

[5] Guinness World Records. 2025. Best-Selling Videogame. Minecraft, 350 million units sold; accessed 2026-08-01. https://www.guinnessworldrecords.com/world-records/best-selling-video-game

[6] Per Landin. 2023. What Is Minecraft? Accessed 2026-08-01. https://www.minecraft.net/en-us/article/what-minecraft

[7] Modrinth. n.d.. Get a Version: Modrinth API Documentation. Accessed 2026-07-21. https://docs.modrinth.com/api/operations/getversion/

[8] Modrinth. n.d.. Modpacks on Modrinth. Accessed 2026-07-21. https://support.modrinth.com/en/articles/8802250-modpacks-on-modrinth

[9] Modrinth. n.d.. Modrinth Modpack Format (`.mrpack`). Accessed 2026-07-21. https://support.modrinth.com/en/articles/8802351-modrinth-modpack-format-mrpack

[10] Modrinth. n.d.. Repairing an Instance. Accessed 2026-07-21. https://support.modrinth.com/en/articles/8797637-repairing-an-instance

[11] Mojang Studios. n.d.. Mods for Minecraft: Java Edition. Accessed 2026-07-21. https://help.minecraft.net/hc/en-us/articles/4409139065613-Mods-for-Minecraft-Java-Edition

[12] packwiz contributors. n.d.. Adding Mods. Accessed 2026-07-21. https://packwiz.infra.link/tutorials/creating/adding-mods/

[13] Donald Pinckney, Federico Cassano, Arjun Guha, Jonathan Bell, Massimiliano Culpo, and Todd Gamblin. 2023. Flexible and Optimal Dependency Management via Max-SMT. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1418–1429. doi:10.1109/ICSE48619.2023.00124

[14] Prism Launcher. n.d.. Mods: Prism Launcher Wiki. Accessed 2026-07-21. https://prismlauncher.org/wiki/help-pages/loader-mods/

[15] Prism Launcher. n.d.. Prism Launcher. Accessed 2026-07-21. https://prismlauncher.org/